UNIVERSIDAD AUSTRAL

INGENIERÍA
Departamento de Computación

APUNTES

Sistemas Distribuidos

Clase 02 – RPC, timeouts, retries e idempotencia

Departamento de Computación

Ingeniería Informática – Comisión A

Pilar – Primer cuatrimestre 2026

Equipo docente

11 de agosto de 2026

1 Introducción

La clase 1 mostró que el problema central de los sistemas distribuidos es la **indistinguibilidad observacional**. La clase 2 estudia cómo esa limitación impacta directamente en el diseño de llamadas remotas.

El objetivo es entender que:

> Una llamada remota no es una función distribuida: es una interacción bajo incertidumbre.

2 RPC: abstracción y límites

RPC (Remote Procedure Call) busca abstraer una invocación remota como si fuera una llamada local. Sin embargo, esta abstracción es incompleta.

2.1 Diferencias fundamentales

- una llamada local no falla parcialmente;
- una llamada remota sí;
- una llamada local tiene semántica clara;
- una llamada remota depende de la red.

3 Modelo de ejecución

Sea una operación remota op.

Eventos posibles:

- envío del request
- recepción en servidor
- ejecución
- envío de respuesta
- recepción en cliente

Cada etapa puede fallar independientemente.

4 No-inmediatez y capas de comunicación

Una comunicación distribuida no debe entenderse como un salto instantáneo entre cliente y servidor. Entre la intención de invocar una operación y la observación de una respuesta hay una trayectoria compuesta por capas: aplicación, librería RPC o cliente HTTP, serialización, transporte, red, balanceadores, servidor, almacenamiento y camino de regreso.

Cada capa puede reaccionar de manera distinta ante una falla:

- la aplicación puede abandonar la espera y mostrar error;
- el cliente RPC puede reintentar automáticamente;

- TCP puede retransmitir segmentos sin que la aplicación lo observe directamente;
- un balanceador puede redirigir a otra instancia;
- el servidor puede seguir procesando aunque el cliente ya haya vencido su timeout;
- la base de datos puede confirmar o abortar una operación independientemente de que la respuesta vuelva.

Esto introduce una forma práctica de concurrencia: mientras una capa decide que "ya esperó suficiente", otra capa puede seguir intentando, entregando, ejecutando o confirmando efectos. Para diagnosticar un problema no alcanza con preguntar "falló o no falló": hay que reconstruir qué observó cada capa, qué acción tomó y qué efecto pudo producir.

La no-inmediatez de la comunicación obliga a razonar con líneas de tiempo: lo que el cliente observa, lo que la red transporta, lo que el servidor ejecuta y lo que el almacenamiento confirma pueden no coincidir en el mismo instante.

5 Semánticas

5.1 At-least-once

El sistema reintenta hasta obtener respuesta.

Ventaja:

- mejora liveness

Problema:

- múltiples ejecuciones

5.2 At-most-once

Evita duplicación.

Problema:

- puede perder ejecución

5.3 Exactly-once

En sistemas distribuidos reales, exactly-once requiere coordinación fuerte o se implementa como combinación de at-least-once + idempotencia.

6 Timeouts

Un timeout define cuándo el cliente deja de esperar.

6.1 Propiedades

- decisión local
- basada en tiempo

- introduce incertidumbre

7 Retries

Retry = reintento tras timeout.

7.1 Problema estructural

Retry introduce duplicación potencial porque:

- no se sabe si la ejecución ocurrió
- se reintenta igual

8 Idempotencia

Una operación es idempotente si múltiples ejecuciones producen el mismo estado final.

8.1 Formalización

$$op(op(s)) = op(s)$$

8.2 Implementación

- request ID
- log de operaciones
- cache de resultados

9 Trade-offs

- idempotencia $\rightarrow$ más estado
- retries $\rightarrow$ más robustez
- timeouts $\rightarrow$ más incertidumbre

10 Conclusión

El diseño de RPC consiste en elegir explícitamente:
- semántica de ejecución
- política de timeout
- política de retry
- mecanismo de idempotencia

11 Síntesis razonada de la clase

Esta unidad debe leerse como parte de una cadena de decisiones de diseño y no como un catálogo de mecanismos. Para estudiar el tema con profundidad conviene reconstruir cuatro preguntas: *qué*

problema aparece, *qué garantía se desea*, *qué mecanismo la aproxima* y *qué costo introduce*. En cada ejemplo debe identificarse además el modelo de falla implícito.

> Una respuesta técnicamente madura no termina al nombrar un algoritmo o patrón: declara sus supuestos, la propiedad que preserva y el escenario en el que deja de ser suficiente.

12 Bibliografía guiada y ruta de profundización

Para profundizar esta clase se recomienda: Kleppmann, cap. 8; Birrell y Nelson sobre RPC; Google SRE sobre timeouts y retries.

La lectura debe hacerse con preguntas concretas: (1) ¿qué modelo de sistema asume el autor?, (2) ¿qué propiedad demuestra o persigue?, (3) ¿qué falla o anomalía motiva el mecanismo?, y (4) ¿qué trade-off queda abierto?

13 Conexión con el Trabajo Final Integrador

El grupo debe señalar qué parte de su sistema utiliza los conceptos de esta unidad, justificar por qué son necesarios y documentar una alternativa descartada. Cuando el contenido todavía no corresponda a la etapa vigente, debe registrarse como hipótesis o decisión pendiente.

14 Autoevaluación conceptual

1. ¿Cuál es el problema distribuido concreto que motiva esta clase?
2. ¿Qué propiedad de seguridad y qué propiedad de progreso están en juego?
3. ¿Qué supuesto de red, tiempo o falla resulta decisivo?
4. ¿Qué trade-off aceptaría en un sistema real y cuál no aceptaría en un dominio crítico?
5. ¿Cómo observaría experimentalmente que la solución se comporta como espera?

15 Profundización autocontenida v9

15.1 Problema, límite e invariante

El núcleo de esta clase es RPC, deadlines, timeouts, retries, backoff, jitter, idempotencia y semánticas de ejecución. Para estudiarlo de manera autónoma conviene separar tres planos. Primero, el *problema observable*: qué comportamiento incorrecto puede ver un cliente o un operador. Segundo, el *límite del modelo*: qué información, sincronía o coordinación no está disponible. Tercero, el *invariante*: qué propiedad no debe violarse aunque el sistema pierda progreso temporalmente.

El modelo de fallas relevante incluye request o respuesta perdida, timeout falso, retry duplicado y sobrecarga. Una solución debe describirse como protocolo y no solo como nombre de tecnología: actores, estados, mensajes, condición de éxito, condición de aborto y estado visible después de una interrupción. La evidencia esperada es traza de requests, IDs idempotentes y medición de éxito/duplicación.

15.2 Método de análisis

Ante un caso nuevo, siga esta secuencia:

1. Identifique actores, dato o recurso compartido y frontera de confianza.
2. Escriba una ejecución nominal y una ejecución adversarial.
3. Distinga seguridad (lo malo no ocurre) de progreso (algo bueno finalmente ocurre).
4. Declare qué supuesto permite avanzar: mayoría, deadline, sincronía parcial, operación idempotente, almacenamiento durable u otro.
5. Compare una alternativa más simple y una más fuerte; registre qué métrica o experimento haría cambiar la decisión.

15.3 Caso transferible

Considere una plataforma de reservas que recibe operaciones duplicadas, pierde conectividad entre regiones y debe sostener un camino crítico sin ocultar el estado real al usuario. Aplique el mecanismo de esta clase a una única decisión concreta. Una respuesta madura no intenta resolver todo: delimita la operación, formula la garantía y explica la degradación esperada cuando el supuesto central deja de cumplirse.

15.4 Errores de razonamiento frecuentes

- confundir una propiedad con el producto que suele implementarla;
- describir el camino feliz sin recorrer una falla intermedia;
- afirmar "exactly-once", "alta disponibilidad" o "consistencia" sin definir alcance observable;
- medir solo throughput aunque la hipótesis trate sobre semántica o recuperación;
- presentar la complejidad añadida como beneficio sin compararla con la alternativa más simple.

16 Lectura guiada

Kleppmann, capítulos 8 y 9; documentación de gRPC/HTTP sobre deadlines e idempotencia.

Durante la lectura, registre: modelo de sistema, propiedad perseguida, contraejemplo que motiva el mecanismo, costo principal y una condición bajo la cual no lo elegiría. Estas cinco anotaciones convierten la bibliografía en una herramienta de diseño y no en una lista para memorizar.

17 Aplicación al Trabajo Final Integrador

El equipo debe producir un ADR breve con la decisión asociada a esta clase. Debe contener: contexto, dos alternativas, supuesto decisivo, garantía elegida, trade-off, evidencia pendiente y fecha de revisión. La evidencia mínima esperada para esta unidad es: traza de requests, IDs idempotentes y medición de éxito/duplicación.

18 Autoevaluación avanzada

1. ¿Puedo explicar la clase sin nombrar primero una tecnología?
2. ¿Puedo construir una ejecución que viole la garantía si el mecanismo se configura mal?
3. ¿Puedo distinguir seguridad, progreso y experiencia del usuario?

4. ¿Puedo proponer una medición que valide la propiedad, no solo el rendimiento?
5. ¿Puedo defender cuándo una solución más simple sería suficiente?